pytotune: An Efficient Open-Source Auto-Tune Python Library

Algorithm Engineering 2026 Project Paper

Hannes Spitz
Friedrich Schiller University Jena
Germany
hannes.spitz@uni-jena.de

Moritz Seppelt
Friedrich Schiller University Jena
Germany
moritz.seppelt@uni-jena.de

Abstract

Pitch correction (Auto-Tune) is a common signal-processing task in music production that requires accurate pitch estimation and modification of audio signals. In this work we investigate how such a system can be engineered as an efficient and reusable open-source library. We present `pytotune`, a Python library with a C++ backend that implements a full pitch-correction pipeline consisting of YIN-based pitch detection, target pitch determination via MIDI or musical scales, and Fourier-based pitch shifting. Through profiling and optimization we improved the performance using parallelization, SIMD vectorization, and signal decimation. Our experiments show that these optimizations substantially reduce the cost of pitch detection and shift the runtime bottleneck to the correction stage.

Keywords

pitch correction, pitch detection, YIN algorithm, audio signal processing, algorithm engineering, performance optimization

1 Introduction

Pitch correction, commonly known as Auto-Tune, is a signal processing task in which the fundamental frequency of an audio signal is estimated and adjusted to match a predefined musical target. Formally, given a time-domain audio signal, the system must extract its pitch trajectory and modify the audio signal, such that its new pitch matches a discrete target frequency defined by a MIDI sequence or by a musical scale. This problem combines frequency estimation, discrete mapping, and signal transformation within a single algorithmic pipeline. While Python provides powerful tools for audio analysis, there are comparatively few open-source libraries that offer an integrated, installable pitch correction solution with a performance-oriented backend and flexible tuning support. One example might be `py-autotune` [7], although it does not seam to be developed with performance in mind.

In this project, we develop an open-source pitch correction library with a C++ core and a Python interface, designed for future distribution via PyPI. The goal is to provide a reusable and accessible tool that integrates efficiently into Python-based audio workflows while leveraging the computational advantages of C++ for the core processing steps.

The system supports two modes of operation. In the first mode, an audio file is corrected according to a corresponding MIDI file, allowing explicit control of the target pitch over time. In the second mode, detected pitches are quantized to a user-defined scale relative to a reference tuning (e.g., A4 = 440 Hz). The design allows flexible pitch sets, including scales that do not necessarily repeat within a single octave, thereby generalizing beyond conventional 12-tone systems.

The focus of this work lies in the systematic engineering of the algorithmic pipeline, while the quality of the result was not the primary concern. We evaluated the implementation empirically to identify performance bottlenecks. Based on these measurements, critical sections were refined and optimized within the C++ backend, while maintaining a clean Python API. This iterative process of implementation, measurement, and targeted improvement reflects the core principles of algorithm engineering.

1.1 Related Work

Pitch correction has been studied extensively in both industry and academia due to its widespread application in music production [5]. Commercial plugins such as Antares Auto-Tune [1] have become standard tools in professional studios, implementing complex pitch estimation and correction strategies that are optimized for perceptual quality and real-time performance. In addition to traditional signal-processing techniques, recent research has explored machine learning approaches to pitch tracking and correction, including deep neural networks [20] trained to estimate fundamental frequency or directly synthesize pitch-shifted audio. These data-driven methods can achieve impressive accuracy in challenging acoustic conditions, but they often sacrifice transparency and interpretability, making it difficult to analyze or adapt their internal behavior without access to training data and model details.

In contrast to data-driven systems, classic algorithmic methods such as autocorrelation [18], cepstral analysis [13], and spectral estimation [19] remain foundational in fundamental frequency detection. Among these, the YIN algorithm [4] is a well-validated approach [17] that estimates pitch by minimizing a difference function derived from the autocorrelation of the signal, providing good stability across a wide range of monophonic inputs.

For the pitch-shifting step, there exist multiple strategies ranging from time-domain resampling [10] to phase vocoder techniques [6]. Stephan Bernsee's Fourier-transform-based [3] method implements pitch shifting by manipulating spectral bins while preserving phase relationships, offering a practical compromise between audio quality and computational cost.

Our project targets a middle path between highly engineered commercial products and opaque machine learning models. We wanted a solution whose behavior we can trace end-to-end and whose performance characteristics we can measure and optimize systematically. To that end, we selected the YIN algorithm for pitch detection and Bernsee's Fourier-based pitch shifter for pitch modification. These choices reflect a preference for algorithmic transparency, reproducibility, and the ability to reason precisely about

AEPRO 2026, March 1, Jena, Germany. Copyright ©2026 for this paper by its authors. Use permitted under Creative Commons License Attribution 4.0 International (CC BY 4.0).

the processing pipeline, which supports our goals of trackability and open-source distribution.

1.2 Outline

The paper is structured as follows. First, we introduce our pitch correction algorithm and describe each stage of the processing pipeline. Next, we highlight the points in the algorithm where profiling revealed optimization opportunities, specifying which were implemented and which were deemed non-critical due to low impact on overall performance. This is followed by a presentation of our experimental results, including measurements of the effectiveness of implemented optimizations and a detailed bottleneck analysis. Finally, we summarize our findings and conclusions.

2 Mode of Operation

The development process followed an agile-inspired workflow with loosely structured sprints complemented by regular meetings to coordinate tasks and track progress. Algorithmic components were initially implemented in a straightforward manner, then profiled and optimized iteratively. We were able to benchmark the performance improvements across macOS and Linux.

Profiling tools such as Intel VTune Profiler [9] and Xcode Instruments [2] were employed to identify computational bottlenecks. Targeted optimizations were subsequently applied to critical sections of the C++ backend, improving execution efficiency while maintaining correctness.

Testing was conducted using the Google Test framework (GTest) [8] to ensure functionality and stability throughout the development cycle and on the different plattforms Windows, MacOS and Linux. Build configuration and compilation were managed with CMake, while Python bindings were implemented using pybind11 [16] to provide a seamless interface. Parallelization opportunities and other optimization stratigies were explored using OpenMP and vectorization using Highway [11].

A small note on quality. We did not do any formal checks for the quality of the output audio, as the goal of this assignment was to implement an optimized algorithm, not to produce a high-quality pitch correction. We still tried to make the output sound as good as possible, and at least for matching a scale (and thus not having a huge correction factor) the output sounded quite good. For larger correction factors, as relevant for the MIDI mode, the output sounded quite robotic. Future improvements could be made by implementing more sophisticated pitch correction algorithms, such as the one described by Laroche and Dolson [10].

3 The Algorithm

3.1 The Pipeline

The processing pipeline begins by reading the input audio file and, if applicable, a corresponding MIDI file. The audio is then divided into short windows, and the YIN algorithm is applied to estimate the pitch for each segment (see 3.3). Target pitches are determined either from the MIDI data or a user-defined scale. Finally, each window is pitch-corrected using Bernsee's Fourier-based algorithm (see 3.4), and the corrected segments are overlapped and combined to synthesize the final output signal.

A basic concept that we pass through the whole pipeline is `Windowing`. This represents the assignment of samples of the sound file to overlapping sections, which we call windows. Typically, one value is assigned to multiple adjacent windows. All pitch detection and shifting is then performed on this grid. The overlap is important for smoothing out the edges when the windows are recombined. For that Hann Windows are used [15, Chapter 13.4, p. 657]. Our default windowing has a width of 4096 samples (at a typical sample rate of 44.1 kHz, this represents 92.9 ms) and a stride of 1024 values.

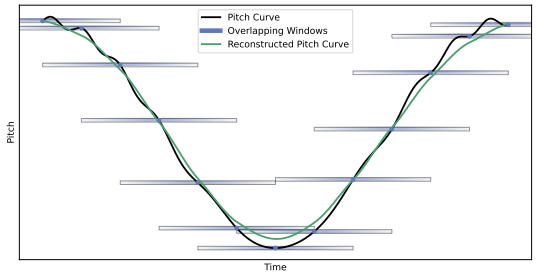


Figure 1: Illustration of the overlapping windowing and pitch reconstruction. Window opacity indicates each window's contribution to the reconstructed pitch using Hann windows.

3.2 Input Processing

3.2.1 File formats. Standard implementations were used to read and write audio and MIDI files without any special optimization. WAV files were loaded to extract raw sample data and MIDI files were parsed to obtain note events and timing information. This information was then converted into windowed data, i.e. which note plays at a particular window. This step contributes negligibly to the program's overall runtime. (see Table 1), and thus did not warrant further optimization.

3.2.2 Scale. We represent a musical scale by the pitch of the base note b, a finite set of factors Σ, so that every note in one part segment of the scale becomes $b \cdot \sigma, \sigma \in \Sigma$, and the factor r when a segment of the scale repeats. The latter is most commonly set to 2, representing one octave. Remember that pitches live on a logarithmic scale, so all distances are multiplicative. Thus, the set of all notes in scale S becomes:

$$S = \{r^k \cdot b \cdot \sigma \mid k \in \mathbb{Z}, \sigma \in \Sigma\}$$

with k, the index of the segment (for $r = 2$ the index of the octave).

The algorithmic problem we had to face, was given some pitch f find the closest pitch $f' \in S$, basically projecting f on S. The projection is given by

$$\pi(f) = \operatorname*{argmin}_{f' \in S} \max \left\{ \frac{f}{f'}, \frac{f'}{f} \right\} = \operatorname*{argmin}_{f' \in S} |\log f - \log f'|$$

We solved this by first determining the octave that f is in, and then using a linear search to obtain the neighbouring notes. Finally, we compared the distances and returned the closer note. A binary search would also be possible, offering a theoretical improvement, but as most scales include fewer than twenty notes, the practical improvement would be negligible.

3.3 Pitch Detection

3.3.1 The YIN Algorithm. The YIN algorithm [4] estimates the fundamental frequency f of a signal by exploiting the near-periodicity of pitched sound. Instead of estimating f directly, we instead estimate a period $\tau^* = s/f$, where s is the sample rate. If a signal $x[n]$ has period τ^*, then $x[n] \approx x[n+\tau^*]$ for all n. This motivates defining the *difference function*

$$d(\tau) = \sum_{j=0}^{W-1-\tau} \left(x[j] - x[j+\tau]\right)^2,$$

where W is the window length. For the true period τ^*, d attains a value close to zero. The pitch estimate is therefore obtained by searching for the lag τ that minimizes $d(\tau)$ within a plausible range $[\tau_{\min}, \tau_{\max}]$. Thus this part of the algorithm is paramaterized by the expected pitch range $[f_{\min}, f_{\max}]$, which translates to $[\tau_{\min}, \tau_{\max}] = [s/f_{\max}, s/f_{\min}]$.

A naive search for the global minimum of d is prone to octave errors, since d also vanishes at integer multiples of τ^*. To address this, YIN introduces the *Cumulative Mean Normalized Difference* (CMND):

$$d'(\tau) = \begin{cases} 1 & \tau = 0, \\ \dfrac{d(\tau)\cdot\tau}{\sum_{j=1}^{\tau} d(j)} & \tau > 0. \end{cases}$$

By normalizing $d(\tau)$ relative to the running mean of all preceding values, d' suppresses the trivial minimum at $\tau = 0$ and penalizes shorter lags, thereby reducing susceptibility to subharmonic errors.

The pitch candidate is selected as the *first local minimum* of d' in $[\tau_{\min}, \tau_{\max}]$ whose value falls below a fixed confidence threshold θ (we use $\theta = 0.4$ per default). If no such minimum is found, the global minimum in the search range serves as a fallback.

To obtain sub-sample accuracy, the selected integer lag $\hat{\tau}$ is refined by fitting a parabola to the three neighboring CMND values and taking its vertex:

$$\hat{\tau}_{\text{refined}} = \hat{\tau} + \frac{d'[\hat{\tau}-1] - d'[\hat{\tau}+1]}{2\big(d'[\hat{\tau}-1] - 2\,d'[\hat{\tau}] + d'[\hat{\tau}+1]\big)}.$$

The fundamental frequency is then recovered as $f = s/\hat{\tau}_{\text{refined}}$, where s denotes the sample rate.

In practice, the per-window pitch estimates can exhibit spurious octave jumps. We address this with two lightweight post-processing steps applied to the pitch trajectory: first, any window whose estimated pitch drops by more than a perfect fifth relative to its predecessor (factor $< 1/1.5$) is assumed to be an octave error and its value is doubled; second, a median filter of width five is applied across the trajectory to suppress remaining isolated outliers without smearing gradual pitch contours.

3.3.2 Optimizations. The dominant cost of YIN is the evaluation of $d(\tau)$ for every lag and every window, an $O(W \cdot (\tau_{\max} - \tau_{\min}))$ operation. We applied three targeted optimizations to reduce this cost in practice.

Parallelization. Each window is processed independently — the difference function, CMND computation, and minimum search for window k do not depend on any other window. We exploit this by distributing the loop over windows across all available CPU cores using an `OpenMP` `parallel for` directive.

Vectorization. Within each window, the inner summation $\sum_j (x[j] - x[j+\tau])^2$ is a straightforward fused subtract-square-accumulate pattern that maps directly to SIMD instructions. We use the Google Highway library [11] to load vectors of consecutive samples, compute the squared difference in a single vectorised expression, and accumulate the result. Highway selects the widest available SIMD instruction set at compile time (e.g. SSE4, AVX2, AVX-512, or NEON), so the same source code benefits from wider registers on newer hardware without manual dispatch.

Downsampling. Pitch detection does not require the full audio bandwidth; only frequencies up to the maximum expected pitch are relevant. We therefore decimate the signal by a factor δ before running YIN, reducing both the number of samples per window and the required lag range, which reduces the cost of evaluating d by approximately a factor of δ^2. To avoid aliasing, we apply a zero-phase FIR low-pass filter prior to downsampling: the signal is convolved with a 101-tap Hamming-windowed sinc filter in both the forward and backward directions, yielding a linear-phase response with no phase distortion [12, 14]. The convolution itself can be parallelised across output samples with `OpenMP` and vectorised with Highway, using a pre-reversed kernel to turn the reverse signal access into a forward walk that SIMD loads can exploit directly. The detected pitches are subsequently mapped back to the original window grid via linear interpolation.

3.4 Pitch Correction

In 3.3 we detected some pitch f_m for each window m, and dependent on the mode, we can extract the target pitch f'_m from the midi file or by projecting onto a scale (see 3.2.2). Now it is left to shift up the pitch of the window by $\alpha_m := \frac{f'_m}{f_m}$.

3.4.1 The Algorithm. We took the algorithm from Staphan Bernsee's Blog on *Pitch Shifting Using The Fourier Transform* [3]. As he already discribed the algorithm in detail, let's just revisit the most important deatils.

The core idea of the pitch shifting algorithm is conceptually simple: each segment of the signal is transformed to the frequency domain using a descrete Fourier transform (DFT), the frequencies are scaled by the desired pitch factor α_m, and then the signal is reconstructed with an inverse Fourier transform. In other words, for a windowed frame $x[n]$, one could naively attempt

$$\tilde{x}[n] = \mathrm{DFT}^{-1}\left(\mathrm{DFT}(x[n])\big|_{k\mapsto\alpha_m k}\right)$$

However, directly scaling frequency bins in this way ignores the phase relationships between successive frames. Here, each *bin* corresponds to a discrete frequency component of the Fourier transform, representing the amplitude and phase of a narrow frequency band. Simply reassigning magnitudes to new bins would produce discontinuities in the waveform, resulting in strong artifacts such as beating or phasiness.

To avoid this, the algorithm keeps track of the instantaneous phase of each frequency component. Let $X_m[k] = |X_m[k]|e^{j\phi_m[k]}$ be the DFT of the m-th windowed frame. The phase difference between successive frames,

$$\Delta\varphi_m[k] = \varphi_m[k] - \varphi_{m-1}[k],$$

is computed and compared against the expected phase advance for that bin based on the window stride. This allows the true instantaneous frequency to be estimated:

$$\omega_m[k] = \frac{2\pi k}{W} + \frac{\Delta\varphi_m[k] - 2\pi k R/W}{R},$$

which represents the actual frequency content rather than the discrete bin location. Here R and W are the window stride and length, respectively.

By scaling these instantaneous frequencies instead of naïvely moving bins, the algorithm preserves temporal continuity. The scaled magnitudes are then reassigned to the corresponding bins in the synthesis frame, the phases are updated accordingly, and the inverse DFT reconstructs the time-domain signal.

3.4.2 Optimizations.

Recursive Twiddles. In our FFT implementation, computing the twiddle factors $W_k = e^{-2\pi i k/N}$ for each butterfly stage can be computationally expensive due to repeated cos and sin calls. This algorithmic optimization replaces direct evaluation of these trigonometric functions with a recurrence relation:

$$W_1 = e^{-2\pi i/N} \qquad W_{k+1} = W_k \cdot W_1$$

This reduces the number of expensive cos/sin evaluations from $O(n \log n)$ to $O(\log n)$ per FFT stage.

Reimplemented Windowing. Bernsee's original implementation of the per window processing made heavy use of an input and output queue, making it difficult to parallelize the code as subsequent iterations depend on the output of the previous one. We reimplemented the windowing and overlap-add procedure to allow for parallel processing of the FFT of the windows. But as described in 3.4.1, the real phase and magnitude of a window still depends on the detected values of the previous one. This part of the code thus has to run sequentially. Although the sequential part of the code has a runtime of $O(n)$, it is still quite heavy, rendering this optimization less effective, see Figure 3.

4 Experiments

All computations were performed on a TUXEDO InfinityFlex 14 Gen1 laptop equipped with an Intel Core i5-1335U processor and 32 GB RAM, running Linux (kernel 6.18.13). The implementation was additionally validated on a MacBook Air 13" (M2) with 16 GB RAM, running macOS Tahoe 26.0.1. Similar results were obtained on both platforms; however, we report the Linux results for consistency.

4.1 Bottleneck Analysis

When we had the algorithm up and running we did early timing tests, to see which parts would be most worth it to improve. We got the timings seen in Table 1.

Thus, we primarily identified the pitch detection as the major bottleneck, which is why we focused our optimization efforts on that part of the code.

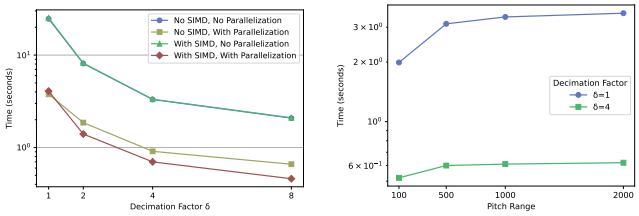
After all optimization were implemented, we reversed the share in the execution time of detection and correction, see Table 2.

Table 1: First bottleneck analysis on a 15 s **sound file, with a detection range of** [20 Hz, 2 kHz]**.**

Program Section	Runtime	Percentage
Total	11.29 s	100 %
Read	804.7 µs	0.0 %
Pitch Detection	10.81 s	95.7 %
Pitch Correction	0.47 s	4.2 %
Write	12.3 µs	0.0 %

Table 2: Bottleneck analysis after all optimizations were implemented, on a 10 min **sound file,** $\delta = 4$**, with a detection range of** [82.41 Hz, 1.047 kHz]**, the range of the human voice. I/O was not separately measured.**

Program Section	Runtime	Percentage
Total	8.83 s	100 %
Pitch Detection	0.62 s	7.6 %
Pitch Correction	8.20 s	92.5 %
Read / Write	—	0.0 %



(a) Execution time for different decimation factors δ. The human pitch range was used.

(b) Execution time for different pitch ranges $f_{max} - f_{min}$. All runs without further optimizations.

Figure 2: Execution time for pitch detection of a 10 min **sound file.**

4.2 Optimizing the Pitch Detection

The performance measurements for the pitch detection stage are shown in Figure 2. Parallelization provides the largest improvement, increasing throughput by nearly a factor of four. This speedup is consistent with the available hardware parallelism and indicates that the workload can be distributed effectively across multiple cores. Within the parallelized implementation, SIMD vectorization yields an additional improvement of approximately 35 %, demonstrating that data-level parallelism further reduces the per-sample processing cost.

Figure 2a also highlights the strong influence of the decimation factor δ. Across all optimization strategies, runtime scales approximately inversely with δ, since increasing the decimation factor reduces the number of processed samples proportionally. As a result, configurations with larger δ values consistently exhibit lower runtimes, while smaller values lead to significantly higher computational cost, while the computational overhead of the downsampling process itself remains negligible in comparison.

A big impact on the performance of the pitch detection is also the pitch range, as shown in Figure 2b, as the number of lags τ that need to be evaluated grows with the pitch range. For low decimation factors, this effect is more pronounced, since the cost of evaluating $d(\tau)$ dominates the runtime. For higher decimation factors, the reduced number of samples per window mitigates this effect, leading to a less steep increase in runtime as the pitch range expands.

4.3 Optimizing the Pitch Correction

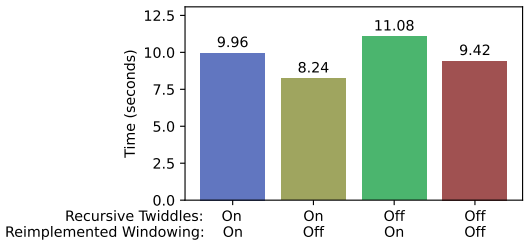


Figure 3: Runtime of our pitch correction implementation for a sound file of 10 min **length for different enabled optimization strategies.**

The results in Figure 3 show that the recursive twiddle optimization improves the overall runtime by approximately 12 %. In contrast, the reimplemented windowing approach leads to a slowdown of about 20 %. While the redesign enables parallel processing of the FFTs of individual windows, a substantial sequential component remains due to dependencies between consecutive windows. As a result, the overhead introduced by parallelization outweighs the benefits of concurrent execution, leading to a net performance decrease.

4.4 The Impact of the Windowing

Quality wise it is not trivial to select an optimal windowing. A small window size allows for a more precise tracking of the pitch, but it also makes the algorithm more susceptible to noise. A large window size, on the other hand, is more robust to noise, but it also makes the algorithm less responsive to changes in pitch. Thus, there is a trade-off between precision and robustness when selecting the window size, though for the sizes we tested, the robustness was the bigger issue, thus preferring larger window sizes.

The runtime of the pitch detection stage is heavily effected by the window size, as shown in Figure 4. Although an increase in window size leads to a decrease in the number of windows, the super-linear runtimer of the YIN algorithm dominates the overall runtime, resulting in a significant increase as the window size grows.

The pitch correction seams to be less affected by the window size, as the runtime of the pitch correction is dominated by the FFT, which has a runtime of $O(n \log n)$, where n is the window size. Thus, the increase in runtime is not as pronounced as for the pitch detection stage.

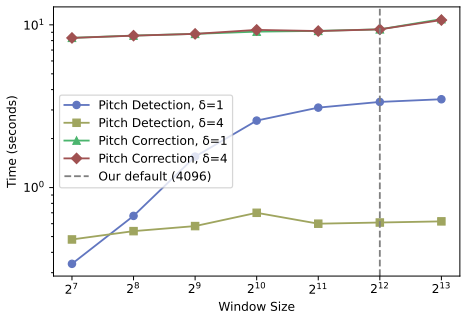


Figure 4: Runtime of our algorithms for a sound file of 10 min **length for different window sizes, with no optimizations enabled.**

5 Conclusions

In this project we developed `pytotune`, an open-source pitch correction library combining a C++ backend with a Python interface for convenient integration into Python-based audio workflows. The system implements a complete pipeline consisting of pitch detection with the YIN algorithm, target pitch determination from MIDI or musical scales, and pitch shifting using a Fourier-transform-based method.

The main focus of this work was the algorithm engineering process. Early profiling revealed pitch detection as the dominant computational bottleneck. By applying parallelization with OpenMP, SIMD vectorization through the Highway library, and signal decimation, we significantly reduced the runtime of this stage. After these optimizations, the computational cost shifted to the pitch correction stage, illustrating how removing one bottleneck can expose the next limiting component.

Our experiments further demonstrate that parameters such as window size, pitch range, and decimation factor strongly influence runtime. Some optimizations that appeared promising in theory did not yield improvements in practice due to remaining sequential dependencies, highlighting the importance of empirical performance analysis.

Overall, from the project we learned the importance of systematic profiling when engineering algorithmic systems. While the theoretical complexity of the individual components was known, the practical bottlenecks only became clear through measurement. In particular, we saw how parallelization and vectorization can substantially reduce the cost of computationally intensive signal-processing tasks, while other seemingly promising optimizations may provide little or even negative benefit due to hidden sequential dependencies.

References

[1] Antares Audio Technologies. 2024. Auto-Tune. https://www.antarestech.com/. Accessed: 2026-02-21.
[2] Apple Inc. 2026. Instruments: Performance Analysis Tool for Xcode. https://developer.apple.com/xcode/. Profiling and tracing tool included with Xcode.
[3] Stephan M. Bernsee. 1999. Pitch Shifting Using The Fourier Transform. https://blogs.zynaptiq.com/bernsee/pitch-shifting-using-the-ft/. Accessed: 2026-02-21.
[4] Alain De Cheveigné and Hideki Kawahara. 2002. YIN, a fundamental frequency estimator for speech and music. *The Journal of the Acoustical Society of America* 111, 4 (2002), 1917–1930.

[5] Joe Diaz. 2009. The Fate of Auto-Tune. *Music and Technology* (2009).
[6] James L Flanagan and Roger M Golden. 1966. Phase vocoder. *Bell system technical Journal* 45, 9 (1966), 1493–1509.
[7] Gabriele Turelli. 2025. py-autotune: Python library for automatic pitch correction. https://pypi.org/project/py-autotune/. PyPI package, accessed: 2026-02-21.
[8] Google LLC. 2026. GoogleTest: C++ Testing Framework. https://github.com/google/googletest. Unit testing and mocking framework for C++.
[9] Intel Corporation. 2026. Intel VTune Profiler. https://www.intel.com/content/www/us/en/developer/tools/oneapi/vtune-profiler.html. Performance profiling and analysis tool.
[10] Jean Laroche. 2002. Time and pitch scale modification of audio signals. In *Applications of digital signal processing to audio and acoustics*. Springer, 279–309.
[11] Google LLC. 2019. Highway, a C++ library that provides portable SIMD/vector intrinsics. https://google.github.io/highway/ Accessed: 2026-02-21.
[12] Richard G. Lyons. 2011. *Understanding Digital Signal Processing*. Pearson.
[13] A Michael Noll. 1967. Cepstrum pitch determination. *The journal of the acoustical society of America* 41, 2 (1967), 293–309.
[14] Alan V. Oppenheim and Ronald W. Schafer. 2010. *Discrete-Time Signal Processing*. Pearson.
[15] William H Press. 2007. *Numerical recipes 3rd edition: The art of scientific computing*. Cambridge university press.
[16] pybind11 developers. 2026. pybind11: Seamless operability between C++11 and Python. https://github.com/pybind/pybind11. Header-only library for creating Python bindings of C++ code.
[17] Lyudmila Sukhostat and Yadigar Imamverdiyev. 2015. A comparative analysis of pitch detection methods under the influence of different noise conditions. *Journal of voice* 29, 4 (2015), 410–417.
[18] Savitha S Upadhya and Nilashree Wankhede. 2012. Pitch estimation using autocorrelation method and AMDF. In *International Conference on Advances in Computing and Management*. 249–253.
[19] Emmanuel Vincent, Nancy Bertin, and Roland Badeau. 2009. Adaptive harmonic spectral decomposition for multiple pitch estimation. *IEEE Transactions on Audio, Speech, and Language Processing* 18, 3 (2009), 528–537.
[20] Sanna Wager, George Tzanetakis, Cheng-i Wang, and Minje Kim. 2020. Deep autotuner: A pitch correcting network for singing performances. In *ICASSP 2020-2020 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 246–250.